SatsPath Threat Model

Disclaimer: This is hackathon software. Do not use with real funds.

Scope

This document covers threats against the SatsPath protocol prototype, focusing on the resolution, signing, and routing layers. It does not cover the underlying Bitcoin, Lightning, or Ark security models.

Threat Table

Threat	Risk	MVP Mitigation	Future Mitigation
Fake alias registration	Attacker registers `alice@bank.com` before Alice	First-come-first-served local registry; no domain verification	Domain-ownership proof via BIP-353 DNS TXT records; DKIM-signed registration challenges
Server/registry tampering	Registry replaces Alice's key/profile	Mandatory signed-event history, checkpoint-bound Merkle inclusion and local checkpoint pinning before quote/pay	Witness gossip and independent monitors
Key replacement attack	Attacker self-signs a profile with a replacement key	Updates reject key changes; rotation requires old authorization plus new acceptance at one canonical sequence	Independent identifier attestations and witnesses
Email takeover	Attacker takes over alice@example.com and re-registers	Registry does not verify email ownership at MVP	DKIM challenge during registration; WebAuthn-based domain binding
LNURL spoofing	Attacker serves a malicious LNURL endpoint	LNURL not verified at MVP; routing is simulated	TLS certificate pinning; LNURL-auth for identity binding; domain-bound LNURL endpoints
On-chain privacy leaks	Multiple payments to the same address reveal transaction graph	Multiple on-chain methods supported; different addresses per method	Silent Payments (BIP-352); hierarchical derivation so each payment gets a fresh address
Lost keys	User loses their `.satspath/keys.json`	Keys are local; demo only; no recovery mechanism at MVP	Nostr-based key backup (NIP-06); seed phrase with BIP-39; multi-sig recovery
Malicious invite links	Attacker crafts an invite URL for a different alias/amount	Invite contains alias hash + amount; receiver should verify independently	Signed invites using sender's identity key; expiry timestamps; single-use tokens

Threat	Risk	MVP Mitigation	Future Mitigation
Ark server trust assumptions	Ark server can censor or delay payments	MockArkClient at MVP; no real Ark integration	Covenants-based Ark with client-side verification; multi-server federation
Fee manipulation	Attacker serves a fake mempool.space response	Falls back to safe `hourFee=5` on API error	Multiple fee data sources; user-configurable fee source; local fee estimation
Replay attacks	Old payment request reused	`updated_at` timestamp in profile; profiles can be revoked by re-signing	Nonce in payment requests; short-lived payment intents with expiry
Profile downgrade	Attacker strips or replaces a method	Authorized removal is permitted but signed, sequenced, logged and displayed; router selects only ownership-verified methods	User policy for change warnings

Trust Model

```
TRUSTED:
  - User's own keypair (generated locally, never transmitted)
  - secp256k1 Schnorr signature validity

## Key transparency threats

A valid self-signature proves control of the key embedded in a profile; it does not prove that a registr

Pins use a stable `log_id`. Presenting a new operator key does not create a new
TOFU namespace: it is a critical error unless a dual-signed operator rotation is
bound to the prior checkpoint. SQLite transactions prevent an event, profile or
checkpoint from advancing alone. Startup replay fails closed if any persisted
checkpoint signature, root, size, predecessor, operator transition or anchor
commitment is corrupt.

PARTIALLY TRUSTED:
  - Initial transparency checkpoint (TOFU until independently compared)
  - mempool.space fee API (falls back to mock on failure)

NOT TRUSTED (at MVP):
  - Unconfigured email/identifier verifiers
  - Domain ownership
  - Ark server
  - LNURL endpoint content
```

Key Principles

1. **Receiver controls keys.** SatsPath never generates or stores keys on behalf of the receiver.
2. **Signatures are mandatory.** No payment should proceed against an unverified profile.
3. **Fail safe.** When in doubt, reject and show a clear error rather than proceeding.
4. **Privacy by default.** Multiple on-chain addresses; avoid address reuse.

5. **Invite rather than proxy.** For unknown users, create an invite that the receiver claims with their own keys.